

CSS Bootcamp, Day 1

Collaborative Foundations

16 September 2026

Agenda

The Reality of Computational Social Science : Before we can run models or wrangle data, we have to survive the installation phase.

For the Next 2 Hours :

1. Base Installations (R, Python, Git).
2. The IDE : Installing and configuring Positron.
3. Authentication : Connecting Git to GitHub.
4. The Command Line : Unix basics for file management.
5. Your First Commit : Testing the workflow locally and remotely.

Goal : By 12 :00, each of you will have a fully reproducible, local coding environment linked to the cloud.

The Core Engines : R and Python

We need the actual programming languages installed on your machine *before* we set up the interface.

1. R Installation :



- ▶ Go to `https://cran.r-project.org/`
- ▶ Download and install the precompiled binary for your OS (Windows, Mac, or Linux).

2. Python Installation :

- ▶ Do **not** just download Python from `python.org`.
- ▶ We will use a package manager to keep environments clean.
- ▶ Go to `https://github.com/conda-forge/miniforge`
- ▶ Download and run the Miniforge3 installer for your OS.

Version Control with Git

Git is the software that tracks changes to your code. It lives on your machine.

Windows Users :

- ▶ Go to <https://gitforwindows.org/>
- ▶ Download and install. **Important** : This also installs "Git Bash," which gives you a Unix-like terminal on Windows. We will use this extensively.

Mac/Linux Users :

- ▶ Open your built-in Terminal application.
- ▶ Type `git -version` and hit enter.
- ▶ If it isn't installed, your Mac will prompt you to install the "Command Line Developer Tools." Click install.



The Positron IDE

Positron is our integrated development environment (IDE). It is the cockpit from which we will write code, run terminals, and manage files.

Installation :

- ▶ Go to <https://github.com/posit-dev/positron/releases>
- ▶ Download the latest release for your operating system.
- ▶ Install and open the application.



Why Positron ? It is natively built for both R and Python data science workflows, seamlessly integrating the command line and Git.

Configuring Git from the Positron Terminal

Now we need to introduce you to Git.

1. Open Positron.
2. Open a new Terminal panel (Terminal > New Terminal). *Windows users : Ensure your terminal is set to Git Bash, not PowerShell.*
3. Type the following commands, hitting Enter after each (replace with your actual name and email) :

```
git config --global user.name "Jane Doe"  
git config --global user.email "jane.doe@ensae.fr"  
git config --global init.defaultBranch main
```

GitHub Authentication

Git (local) needs permission to talk to GitHub (remote). We will use a Personal Access Token (PAT).



1. Log in to `https://github.com`.
2. Go to Settings > Developer Settings > Personal Access Tokens > Tokens (classic).
3. Click **Generate new token (classic)**.
4. Name it "Bootcamp Positron", set expiration to 90 days, and check the **repo** box.
5. Click Generate. Copy this token.

Connecting Git and GitHub in Positron

Storing your credentials so you don't have to type them every time :

In your Positron terminal, run :

```
git config --global credential.helper store
```

The first time you attempt to push code to GitHub (which we will do shortly), it will ask for your username and password.

CRITICAL : When it asks for your password, paste your **Personal Access Token (PAT)** instead of your actual GitHub password.

Unix Basics (The Command Line)

Before we test the Git connection, let's learn to move around our machine using the terminal.

- ▶ `pwd` : Print working directory (Where am I right now ?)
- ▶ `ls` : List directory contents (What is in this folder ?)
- ▶ `cd <folder>` : Change directory (Move into a folder)
- ▶ `cd ..` : Move up one folder level
- ▶ `mkdir <folder_name>` : Create a new folder

Mini-Task : Use these commands to navigate to your Desktop and create a folder called `my_repo`.

Your First Local Repository

Navigate into your new folder via the terminal :

```
cd my_repo
```

Initialize Git, create a file, and make your first commit :

```
git init
echo "# My repo" > README.md
git add README.md
git commit -m "Initial commit: setting up environment"
git status
```

Pushing to the Cloud

1. Go to GitHub.com and create a new, empty repository called `my_repo`.
2. Do NOT check "Add a README" (we just made one locally).
3. Copy the repository URL (e.g.,
`https://github.com/yourusername/my_repo.git`).

Back in your Positron terminal, link them and push :

```
git remote add origin <YOUR_URL>
git push -u origin main
```

(This is where you paste your PAT if prompted for a password !)

Cloning the Course Repository

You now know how to push your own code. Next, we need to download the official materials for this week.

Instead of initializing a new repository from scratch, we will **clone** an existing one from GitHub.

```
# 1. Move back out of your repo folder
cd ..
# 2. Clone the main bootcamp repository
git clone https://github.com/Marshall-Soc/css_bootcamp.git
# 3. Move into the new course folder
cd css_bootcamp
```

This folder now contains the slides, scripts, and datasets (like the ESS extract) you will need for the afternoon exercise.

Alternative GUI : Managing Projects with GitHub Desktop

For those who prefer a visual interface over the terminal, **GitHub Desktop** offers a streamlined GUI for Git and GitHub workflows.

Why use GitHub Desktop for collaborative projects ?

- ▶ **Visual Diff Checker** : Instantly inspect additions (green) and deletions (red) side-by-side before committing.
- ▶ **Simplified Branching** : Switch between branches or create new feature branches without memorizing command-line syntax.
- ▶ **One-Click Syncing** : Fetch, pull, and push updates with dedicated buttons instead of manual CLI commands.

Download at <https://desktop.github.com/> and log in with your GitHub account.

The Collaborative Loop in GitHub Desktop

When working on a shared group project this afternoon, follow this cycle :

1. **Fetch Origin / Pull** : Always click *Fetch origin* and pull latest changes before starting work to stay in sync.
2. **Make Changes & Stage** : Edit files in your IDE (Positron/RStudio) ; GitHub Desktop automatically tracks modified files.
3. **Commit** : Type a brief summary message in the bottom-left box and click *Commit to main*.
4. **Push Origin** : Click *Push origin* to upload your changes back to the shared cloud repository.

Tip : If a teammate pushed conflicting code, GitHub Desktop flags the conflict visually and lets you resolve it cleanly in your editor.

The Collaborative Loop in the Terminal

For those who prefer the command line, here is the Git equivalent :

```
# 1. Pull: Get the latest changes before you start
git pull origin main
# 2. Stage: Tell Git which edited files to track
git add example_file.R
# 3. Commit: Save changes locally with a message
git commit -m "Example commit message"
# 4. Push: Upload to the shared cloud repository
git push origin main
```

Tip : If you get a merge conflict during the pull or push, Positron will highlight the conflicting lines in your script so you can fix them, save, and commit again !



BREAK FOR LUNCH



ENSAE-ENSAI
Groupe
(GENES)



Group Application : Cleaning a Dataset

Follow the instructions here : bit.ly/css_bootcamp_day1.